

Secure Media Streaming Stack

Technical Report: RTSSP & RTSSP-SHP Implementation

Course: Network and Computer Security

Project: Mandatory Project Assignment 1

Authors:

João Ventura (65590)

Miguel Ventura (65928)

May 12, 2026

Contents

1	Project Overview	2
1.1	Addressing Core Security Vulnerabilities	2
2	Control and Instrumentation	3
2.1	Session Management and Signaling	3
2.2	Performance Instrumentation	3
3	Design Specifications	4
3.1	Architectural Components	4
3.2	The Secure Handshake Protocol (SHP) Workflow	4
4	Implementation and Cryptographic Details	5
4.1	Cryptographic Configurations	5
4.2	Initialization Vectors and Counter Modes	5
5	Experimental Results	6
5.1	Comparative Performance Metrics	6
5.2	Analysis of Experimental Observations	6
6	Conclusion	7

1. Project Overview

This report documents the design and implementation of a secure real-time media dissemination architecture. The primary goal was to transition from a baseline system that disseminated media in "clear packets" to a robust, authenticated, and encrypted stack.

The system is developed in two evolutionary phases:

- **Phase 1 (RTSSP)**: Implementation of a secure UDP transport channel using static cryptographic configurations.
- **Phase 2 (SHP)**: Integration of a dynamic **Secure Handshake Protocol** inspired by TLS 1.3 to manage certificate-based mutual authentication and ephemeral key establishment.

2. System Architecture

The system architecture follows a three-tier model designed to isolate the service logic from the delivery and consumption layers.

2.1 Logical Data Flow

The architecture is composed of the Streaming Server, a Middle-Box (Proxy), and the final Media Player. The interaction follows a strict port-mapping convention:

- **TCP Port 8888:** Utilized for the SHP handshake and session negotiation.
- **UDP Port 5000:** The encrypted transport channel for RTSSP data segments.
- **UDP Port 7777:** The local, clear-text delivery channel between the proxy and the player.

2.2 Trusted Computing Base (TCB)

We assume a TCB where the endpoints and runtime environments (Java JVM, OS, and hardware) are trustable. Security is focused exclusively on protecting the communication channel against external adversaries targeting the network environment.

3. Protocol Specifications

3.1 RTSSP Packet Format

The protocol utilizes a specialized packet format inspired by the RLP (Record Layer Protocol) found in TLS 1.2 and 1.3.

Table 3.1: Byte-level RTSSP Packet Structure

Field	Size (Bytes)	Description
Header	4	Version, Type, and Payload Length
IV	16	Per-frame Initialization Vector
Payload	Variable	Encrypted MPEG4 media frame
HMAC	32	HMAC-SHA256 Authentication Tag

3.2 Session Control and Signaling

To provide robust session management, we implemented a set of control messages that were absent in the initial materials:

- **START / FINISH:** These messages manage the initialization and termination of the streaming session, ensuring that both the server and proxy are synchronized.
- **RESPONSE / ERROR:** These allow the server to communicate the availability of a requested movie or report a "Not Found" condition before attempting to stream.

4. Cryptographic Details

4.1 The Secure Handshake Protocol (SHP)

The SHP facilitates a dynamic transition from static keys to ephemeral session keys. It uses a three-step mutual authentication flow:

1. **CLIENT_HELLO**: Proxy sends movie name, ECDH public share, and proposed ciphersuites.
2. **SERVER_HELLO**: Server validates the proxy certificate and returns its own ECDH share and the selected ciphersuite.
3. **CCSP**: The proxy confirms the session by responding to a nonce challenge.

4.2 Algorithm Justification

- **Curve**: We use **secp256r1** (NIST P-256) for its balance of high security and computational efficiency.
- **Signatures**: Handshake messages are signed using **SHA256withECDSA**.
- **Key Derivation**: We implement **HKDF-SHA256** to expand the ECDH shared secret into independent Cipher and HMAC keys.

$$Key_{Session} = \text{HKDF-Expand}(\text{HKDF-Extract}(0, SharedSecret), "encryption", 32) \quad (4.1)$$

5. Security Analysis

5.1 Threat Model and Countermeasures

The stack is designed to defend against a standard networking adversary model:

- **Packet Sniffing:** Neutralized via AES encryption (CTR or GCM modes).
- **Replay Attacks:** Mitigated by the use of random nonces in the handshake and unique per-frame IVs.
- **Integrity Violations:** Detected through constant-time HMAC-SHA256 verification.

5.2 Forward Secrecy

By utilizing ephemeral ECDH keys, the protocol ensures **Perfect Forward Secrecy**. If the server's long-term private key were compromised in the future, past sessions could still not be decrypted because the session keys were never stored or transmitted.

6. Test Environment and Results

6.1 System Environment

- **Runtime:** Java JDK 11+ with JCA/JCE cryptographic providers.
- **Hardware:** Linux Ubuntu 22.04 LTS on a 64-bit architecture.
- **Player:** Celluloid Media Player.

6.2 Performance Metrics

The system was benchmarked using a custom `test-performance.sh` script.

Table 6.1: Experimental Observations

Mode	Ciphersuite	Frames	Avg. FPS	Status
RTSSP	AES/CTR/256	30,515	238	PASS
RTSSP-SHP	AES/GCM/256	25,945	184	PASS
RTSSP-SHP	AES/GCM/128	30,515	238	PASS

7. Usage Instructions

7.1 Initialization and Compilation

The project uses Maven for dependency management and execution. To initialize the cryptographic environment and compile the binaries:

```
1 # Run the automated setup script
2 ./setup.sh
```

7.2 Execution

To start the full stack (Server, Proxy, and Player) in a single command:

```
1 ./run.sh
```

8. Conclusion

The implementation of the RTSSP and SHP stack successfully secures the media dissemination pipeline without compromising real-time performance. By integrating modern cryptographic standards such as ECDH and ECDSA, the system achieves a production-ready security posture that protects against sniffing, tampering, and replay attacks.